

FATEC MAUÁ

Guia de Oficina - Product Backlog do Projeto Integrador

Como transformar problema de negócio, funcionalidades, riscos e requisitos em um backlog utilizável pelo time

Passo a passo, templates, exemplos e critérios de qualidade

Atividade da segunda parte da aula - trabalho em grupo com acompanhamento do docente

Curso Superior de Tecnologia em Desenvolvimento de Software Multiplataforma
ISG-022 - Segurança no Desenvolvimento de Aplicações

1. Objetivo da oficina

Ao final da segunda parte da aula, cada grupo deverá possuir um Product Backlog inicial coerente, priorizado e registrável no GitHub. O backlog deve representar o produto que o grupo pretende construir e também tornar visível o trabalho de segurança derivado dos riscos da disciplina.

Produto antes de tarefa

Product Backlog não é uma lista de “fazer tela”, “criar tabela” e “programar API”. O backlog deve explicar resultados, comportamentos e problemas que o produto precisa resolver; tarefas técnicas podem existir como decomposição.

2. O que precisa existir ao final da aula

- Product Goal em uma frase clara.
- Atores/personas principais.
- De 3 a 6 épicos ou grandes capacidades.
- De 12 a 20 itens iniciais de backlog, compatíveis com a complexidade do projeto.
- Critérios de aceite nos itens prioritários.
- Pelo menos 3 itens explícitos de segurança neste momento, que serão expandidos no laboratório.
- Prioridade inicial e indicação do que pode compor a próxima Sprint.
- Registro no GitHub Issues/Projects ou arquivo Markdown temporário pronto para migração.

3. Antes de escrever histórias: alinhar o Product Goal

O Product Goal descreve o resultado que o produto pretende alcançar. Deve caber em uma frase e ser compreensível por alguém de fora do grupo.

Fraco	Melhor
“Criar um sistema web.”	“Permitir que pequenos prestadores publiquem serviços e que clientes encontrem e solicitem atendimento com rastreabilidade e controle de acesso.”
“Fazer um app de saúde.”	“Permitir que pacientes acompanhem agendamentos e documentos clínicos, com acesso restrito aos perfis autorizados.”

Passo 1 - Escrever o Product Goal

Nosso produto permitirá que [público/atores]
consigam [resultado principal]
para [benefício/necessidade],
respeitando [restrições importantes de negócio/segurança].

O grupo deve validar a frase com todos os integrantes antes de prosseguir.

4. Passo 2 - Mapear atores e permissões

Liste quem usa, administra, opera ou integra com o produto. Em Segurança de Aplicações, atores ajudam a identificar autorização e abuso.

Ator	Objetivos legítimos	Dados/ações	Restrições
Usuário comum	Usar funcionalidade principal	Seus próprios dados e recursos	Não acessa dados de outro usuário
Administrador	Gerenciar operação	Configurações e usuários	Ações críticas auditadas e

			limitadas
Sistema externo	Trocar informação	Dados da integração	Somente escopo necessário; autenticação própria

5. Passo 3 - Identificar épicos

Épico é uma capacidade ampla demais para uma única Sprint e serve para organizar histórias. Exemplos: Identidade e Acesso; Catálogo; Solicitações; Pagamentos; Administração; Auditoria e Segurança.

Teste do épico

Se o item pode ser entregue em poucos dias por uma pessoa e possui critérios de aceite específicos, provavelmente já é uma história/tarefa e não um épico.

6. Passo 4 - Escrever histórias de usuário

Use o formato Como [ator], quero [capacidade/comportamento], para [benefício]. O formato não é obrigatório em toda situação, mas ajuda a evitar backlog técnico sem contexto.

```
US-001
Como cliente,
quero visualizar meus pedidos,
para acompanhar o status das solicitações realizadas.
```

O que uma boa história evita

Ruim	Problema	Reescrita
Criar tabela pedidos	Tarefa técnica sem valor/resultado	Como cliente, quero consultar meus pedidos para acompanhar solicitações.
Fazer login	Vago	Como usuário cadastrado, quero autenticar minha identidade para acessar funcionalidades protegidas.
Implementar segurança	Não testável e amplo	Como usuário, quero que somente perfis autorizados acessem meus recursos para evitar exposição indevida.

7. Passo 5 - Escrever critérios de aceite

Critério de aceite descreve condições objetivas que permitem verificar se o item foi entregue. Uma história sem critério de aceite está aberta a interpretações diferentes.

```
Critérios de aceite - US-001
- Usuário autenticado visualiza somente seus próprios pedidos.
- A lista informa identificador, data, status e valor.
- Usuário sem pedidos recebe estado vazio compreensível.
- Requisição não autenticada é negada.
- Erro de backend não expõe stack trace ou dados internos.
```

Formato Given / When / Then

```
Dado que o usuário A está autenticado
E existe um pedido pertencente ao usuário B
Quando A solicitar o pedido de B
Então a API deve negar o acesso
E não deve retornar dados do pedido.
```

8. Passo 6 - Adicionar segurança ao backlog

Segurança pode aparecer dentro dos critérios de uma história funcional ou como item próprio quando existe trabalho significativo, risco específico, dívida técnica ou decisão arquitetural.

Forma	Quando usar	Exemplo
Critério de aceite	A segurança faz parte intrínseca da funcionalidade	"Usuário vê apenas seus próprios pedidos."
Security Story	Há objetivo de proteção percebido pelo usuário/negócio	"Como usuário, quero que sessões expirem para reduzir risco de uso indevido."
Tarefa/Enabler técnico	Trabalho técnico necessário para habilitar controle	"Implementar policy de autorização por recurso na API."
Bug/Vulnerabilidade	Comportamento existente viola requisito	"Corrigir endpoint que retorna registro de outro usuário."

9. Passo 7 - Ligar risco e requisito ao item

SEC-ISSUE-003 - Validar autorização por recurso

Contexto:

Risco R-01 / Threat T-03: acesso horizontal a dados.

Requisito SEC-01: a API deve validar autorização por recurso.

Critérios de aceite:

- usuário A não acessa recurso de B;
- decisão ocorre no backend;
- tentativas negadas são registradas sem conteúdo sensível;
- testes automatizados cobrem acesso permitido e negado.

Essa rastreabilidade será importante durante a defesa final do projeto: o grupo deve conseguir contar de onde a decisão veio e como foi verificada.

10. Passo 8 - Quebrar itens grandes

Um item deve ser pequeno o suficiente para ser concluído dentro da Sprint. Histórias grandes demais ficam eternamente "em andamento" e escondem risco.

Item grande	Possíveis fatias
Implementar autenticação	Cadastro de credencial; login; logout; expiração de sessão; recuperação de senha; bloqueio/rate limit.
Criar módulo de pedidos	Criar pedido; listar meus pedidos; visualizar detalhe; cancelar; alterar status com regra de papel.
Segurança da aplicação	Segredos; autorização; validação; logs; erros; dependências; testes.

Técnica de divisão

- Por fluxo do usuário.
- Por operação: criar, ler, atualizar, excluir.
- Por regra/variação de negócio.
- Por perfil de acesso.
- Por caminho feliz e depois exceções, sem deixar segurança estrutural para o final.

11. Passo 9 - Priorizar

A ordem do Product Backlog representa decisão. Use valor de negócio, dependências, aprendizado e risco. Uma priorização inicial simples pode combinar MoSCoW com criticidade de segurança.

Prioridade	Uso
Must	Essencial para o incremento/produto ou necessário para reduzir risco inaceitável.
Should	Muito importante, mas existe workaround/adiamento controlado.
Could	Agrega valor, porém não compromete o objetivo imediato.
Won't now	Fora do horizonte atual; registrar para evitar expectativa implícita.

Risco altera prioridade

Uma história funcional pequena não deve automaticamente ficar acima de um requisito que corrige exposição grave de dados. Priorize com justificativa, não apenas por facilidade.

12. Passo 10 - Estimar esforço sem falsa precisão

Para o projeto da disciplina, a estimativa pode ser simples: P/M/G ou Story Points (1,2,3,5,8). O objetivo é discutir tamanho e incerteza, não prever horas exatas.

Sinal	Interpretação
1-3 pontos / P	Pequeno, bem compreendido, poucas dependências.
5 pontos / M	Trabalho moderado, alguma incerteza.
8+ pontos / G	Provavelmente precisa ser quebrado antes de entrar na Sprint.

13. Passo 11 - Dependências e sequência

Marque dependências explícitas. Ex.: “Visualizar meus pedidos” depende de autenticação e de existir um pedido. “Testar autorização” depende de endpoint protegido e regras de acesso definidas. Dependência desconhecida é fonte comum de atraso.

14. Passo 12 - Definition of Ready

Antes de selecionar um item para a Sprint, verifique se ele está pronto para execução.

- Objetivo e contexto compreensíveis.
- Critérios de aceite definidos.
- Dependências conhecidas.
- Impacto de segurança identificado quando aplicável.
- Tamanho compatível com a Sprint ou item decomposto.
- Dúvidas críticas resolvidas ou registradas como hipótese explícita.

15. Passo 13 - Definition of Done

- Implementação/documentação integrada e executável quando aplicável.
- Critérios de aceite atendidos.

- Código/revisão por outro integrante quando relevante.
- Testes relevantes executados.
- Nenhum segredo real versionado.
- Documentação e threat model atualizados quando a mudança impactar segurança.
- Issue/PR contém evidência suficiente para demonstrar a entrega.

16. Passo 14 - Registrar no GitHub

Recomendação: uma Issue por item significativo e um GitHub Project com colunas/status. Estrutura inicial possível: Backlog -> Ready -> In Progress -> Review -> Done.

```
Título: US-005 - Consultar meus pedidos

Descrição:
Como cliente, quero consultar meus pedidos para acompanhar o andamento.

Critérios de aceite:
- ...

Segurança:
- Usuário só consulta recursos que possui autorização.
- Referência: SEC-01 / R-01.

Prioridade: Must
Estimativa: 3
Responsável: @usuario
Sprint: Sprint 1/2
```

Labels sugeridas

- type:story, type:bug, type:task, type:security
- priority:must, priority:should, priority:could
- area:frontend, area:backend, area:data, area:docs, area:security
- status:ready quando não estiver usando Project status

17. Passo 15 - Selecionar a próxima Sprint

Depois de ordenar o backlog, defina um Sprint Goal e selecione um conjunto coerente de itens. Não encha a Sprint com tudo que parece importante; escolha trabalho que forma um incremento demonstrável.

```
Sprint Goal - exemplo
"Entregar o fluxo mínimo de autenticação e consulta de dados próprios,
com regras de autorização definidas e evidências iniciais de segurança."
```

18. Roteiro de 100 minutos para a oficina

Tempo	Atividade do grupo	Saída
0-10	Revisar problema, usuários e Product Goal.	Goal validado.
10-25	Mapear atores e épicos.	3-6 épicos.
25-50	Escrever histórias/itens.	12-20 itens brutos.
50-70	Adicionar critérios de aceite e segurança.	Itens prioritários refinados.
70-85	Quebrar, estimar e priorizar.	Backlog ordenado.
85-95	Definir Sprint Goal e candidatos Ready.	Próxima Sprint proposta.
95-100	Registrar evidência/commit ou GitHub Project.	Checkpoint final.

19. Modelo de backlog em tabela

ID	Épico	Item	Tipo	Prioridade	Estim.	Status
US-001	Identidade	Cadastrar usuário	Story	Must	3	Ready
US-002	Identidade	Autenticar usuário	Story	Must	5	Backlog
SEC-001	Segurança	Autorizar acesso por recurso	Security	Must	5	Backlog
US-003	Core	Consultar meus registros	Story	Must	3	Backlog
DOC-001	Docs	Atualizar arquitetura	Task	Should	2	Backlog

20. Checklist de qualidade antes de encerrar

- ☐ O Product Goal explica resultado/valor?
- ☐ Os atores principais estão identificados?
- ☐ Os épicos organizam o produto sem virar tarefas?
- ☐ Os itens prioritários possuem contexto e comportamento verificável?
- ☐ Há critérios de aceite objetivos?
- ☐ Segurança aparece explicitamente onde existe risco?
- ☐ Itens grandes foram quebrados?
- ☐ Prioridade possui justificativa de valor/risco?
- ☐ Dependências críticas estão visíveis?
- ☐ A próxima Sprint possui Goal e itens Ready?
- ☐ O backlog está registrado em local compartilhado e versionado/rastreável?

21. Anti-padrões que o professor devolverá para revisão

Anti-padrão	Correção esperada
Backlog = lista de tarefas técnicas	Reescrever em capacidades/resultados; manter tarefas como decomposição.
Histórias sem benefício	Explicitar por que o usuário/negócio precisa do comportamento.
Sem critérios de aceite	Adicionar condições verificáveis.
Tudo prioridade alta	Ordenar de verdade; usar valor, dependência e risco.
Segurança em um único item “depois”	Distribuir controles/requisitos nas histórias e criar itens próprios quando necessário.
Épicos enormes sem decomposição	Quebrar em fluxos/operações/perfis/variações.

22. Referências

GitHub Docs - Issues e planejamento de trabalho. - <https://docs.github.com/en/issues>

GitHub Docs - Projects. - <https://docs.github.com/en/issues/planning-and-tracking-with-projects>

OWASP Threat Modeling Cheat Sheet. - https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html

OWASP ASVS 5.0.0 - requisitos de segurança verificáveis. - <https://owasp.org/www-project-application-security-verification-standard/>

ISG022 - Projeto Integrador Segurança de Aplicações, 2026/2.